

Final DevOps Project: One-Click Jenkins Pipeline Deployment

Objective

Build a fully automated DevOps pipeline using Jenkins (in Docker) that:

1. Provisions a VM on Azure using Terraform
2. Installs a web server on that VM using Ansible
3. Deploys a static web app to that server via Ansible
4. Runs all these steps from a single Jenkins pipeline

Technology Stack

Tool	Purpose
Docker	Host Jenkins in a container (no Compose)
Jenkins	Automate the workflow
Terraform	Provision the virtual machine
Ansible	Configure the VM and deploy the web app
Azure	Host the virtual machine
Git	Store Jenkinsfile, code, playbooks, Terraform code

Project Structure

```
project/
├── terraform/
│   ├── main.tf
│   └── variables.tf
├── ansible/
│   └── install_web.yml
├── app/
│   └── index.html
├── Jenkinsfile
└── README.md
```

Jenkins in Docker

Run Jenkins (with Docker and required tools accessible inside):

```
docker run -d \
--name jenkins \
-p 8080:8080 -p 50000:50000 \
-v jenkins_home:/var/jenkins_home \
-v /var/run/docker.sock:/var/run/docker.sock \
-v $(which terraform):/usr/local/bin/terraform \
-v $(which ansible-playbook):/usr/local/bin/ansible-playbook \
-v $(pwd):/workspace \
jenkins/jenkins:lts
```

Post-install steps inside Jenkins:

- Install plugins: Git, Pipeline, SSH Agent
- Add credentials for SSH private key (used by Ansible)

Terraform on Azure

Terraform code provisions an Ubuntu VM on Azure. Files include `main.tf`, `variables.tf`, and optional `terraform.tfvars`.

Ansible Configuration

Installs Apache on the VM and deploys `index.html` to `/var/www/html/`. Uses a dynamic inventory generated from Terraform output.

Jenkinsfile Pipeline

Executes Terraform and Ansible in sequential stages, verifies deployment via curl.

Evaluation Criteria

Task	Points
Jenkins runs from Docker	2
Terraform script creates VM	4
Ansible installs and configures web server	4
Static site deployed correctly	4
Jenkins pipeline triggers and completes all stages	4
Clean repo & documentation	2